



Gustave Eiffel University

**Internship Report**  
Master 2 Bézout

---

# Fine-Grained Analysis of Hyperscan Engine

---

CHAU Dang Minh

**Supervisors:** Prof. Cyril NICAUD (LIGM)  
Dr. Pablo ROTONDO (LIGM)

Paris, July 2026



# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                           | <b>3</b>  |
| <b>1 Introduction</b>                                            | <b>4</b>  |
| <b>2 An Overview of the Hyperscan Engine</b>                     | <b>5</b>  |
| 2.1 Compilation . . . . .                                        | 5         |
| 2.2 Execution . . . . .                                          | 7         |
| 2.3 The FDR Matcher . . . . .                                    | 7         |
| 2.4 Related Problems . . . . .                                   | 8         |
| 2.4.1 Optimizing regular expression decomposition . . . . .      | 8         |
| 2.4.2 Analysis of the FDR string matcher . . . . .               | 9         |
| 2.4.3 Statistics of regular expressions . . . . .                | 9         |
| <b>3 The Length of Longest Literals</b>                          | <b>10</b> |
| 3.1 Lower and upper bounds . . . . .                             | 10        |
| 3.2 Asymptotics of the lower bound recurrence . . . . .          | 11        |
| 3.3 Asymptotics . . . . .                                        | 20        |
| <b>A Appendix</b>                                                | <b>22</b> |
| A.1 The Shift-Or algorithm . . . . .                             | 22        |
| A.2 BST-like distributions of regular expression trees . . . . . | 22        |
| A.3 Basics of Monte-Carlo simulation . . . . .                   | 23        |
| A.4 Remaining proofs . . . . .                                   | 23        |
| A.4.1 Lemma 3.13 . . . . .                                       | 23        |
| <b>B Bibliography</b>                                            | <b>24</b> |

# List of Figures

|     |                                                                                             |   |
|-----|---------------------------------------------------------------------------------------------|---|
| 2.1 | Dependency graph of main functions in the compilation phase. . . . .                        | 6 |
| 2.2 | The dependency graphs by Violet and Rose. . . . .                                           | 6 |
| 2.3 | Dependency graph of main functions in the execution phase. . . . .                          | 7 |
| 2.4 | Proposed sequential dependency graph representation for <code>abc[0-9]+xyz</code> . . . . . | 8 |

# 1

## Introduction

Since Knuth’s announcement of the term *analysis of algorithms* [3, Preface], the field has grown to include highly specialized subfields. A basic classification is based on two aspects. The first aspect is the scenario under which the performance of an algorithm is analyzed, which concludes worst-case and average-case analysis. In general, average-case analysis is more informative than worst-case analysis, and is often more challenging to perform, since it requires a probabilistic model of the input that is tractable, yet expressive enough. For example, in the case of regular languages, it may not be a good idea to generate their syntax trees uniformly at random [4]. Interestingly, average-case analysis opens up the door to apply techniques from other fields, such as probability theory and analytic combinatorics, and raises new questions, such as those about the limiting objects when the input size tends to infinity. The second aspect is the underlying cost model, which can be the uniform or word-RAM model. The main purpose is to align quantitative analysis with practical performance, demonstrated by execution time. Modern analysis of modern algorithms requires new cost models to natively capture hardware features, such as caching, branch prediction and parallelism.

In this internship, we study Hyperscan a regular expression matching engine that leverages SIMD parallelism to achieve high performance. Acknowledging the sophisticated design of Hyperscan, we proposed related questions and attempted to answer them. Hyperscan works by extracting possible strings from a regular expression, and then matching them once in parallel by its SIMD string matchers. Hence it is natural to analyze statistics related to those strings under a distribution. In particular, our question is to understand the asymptotic of the length of the longest strings extracted from a regular expression. Other questions are also posed clearly although not studied to a considerable extent. Details are given in the next chapter.

The rest of the report is organized as follows. In **Chapter 2**, we present Hyperscan and problems arising in the analysis. **Chapter 3** contains our results on the question of the length of the longest strings. Before each proof, we present the motivation and reasoning behind our approaches, which the reader can optionally skip and then go back after reading the proof. We place details of popular theoretical and experimental techniques in **Chapter 4**, where the reader can refer to for background knowledge. Finally, we conclude in **Chapter 5** with a summary of our work and future directions.

# 2

## An Overview of the Hyperscan Engine

The regular expression matching engine Hyperscan [8] was primarily developed by the startup company Sensory Networks in 2008, and was later acquired and open-sourced by Intel in 2013. Closed source in 2023 at version 5.4, it is forked into Vectorscan [7] by VectorCamp to support more architectures like ARM. Since Vectorscan is still being actively maintained publicly, analyzing its core Hyperscan functionality still provides valuable insights into the design and optimization of high-performance regular expression matching engines, interesting to both academia and industry.

Hyperscan is designed to match a set of regular expressions against a stream of data, with its primary use case being deep packet inspection. The engine leverages SIMD (Single Instruction, Multiple Data) features supported by modern CPUs, which uses longer registers to process multiple similar operations in one instruction. Architecturally, it operates in two distinct phases: the compilation phase and the execution phase. It supports two modes of operation: block mode and stream mode. In block mode, the engine processes a single block of data at a time, while in stream mode, it maintains state across multiple blocks, allowing for continuous matching over a data stream. We focus on the block mode.

### 2.1 Compilation

An overview of the compilation process is illustrated in Figure 2.1. The compilation phase provides central APIs, such as the `hs.compile` function. The function accepts a set of regular expressions. A facade object of class `NG` is initialized, which consumes a compile context of type `CompileContext`. Through the function `addExpression`, each expression is converted into a Glushkov NFA [1] (through `buildGraph`) and added to the facade (through `NG.addGraph`).

In `NG.addGraph`, the NFA is firstly decomposed base on its language through `calcComponents`. If its language is  $L$ , it may be split into NFAs whose languages are  $L_1, L_2, \dots, L_n$  such that

$$L = L_1 \cup L_2 \cup \dots \cup L_n.$$

The components are added one by one through `addComponent`, which then passed to the Violet subengine through the `doViolet` function, which in turn calls `doInitialVioletTransform` and `RoseBuild.addRose`.

In `doInitialVioletTransform`, the graph analyses described in the Hyperscan paper—namely dominant path, dominant region, and network flow analysis—are performed. These analyses decompose the NFA

## 2.1 Compilation

into literals and sub-FAs, yielding a dependency graph that dictates the matching order. At this phase (**RoseInGraph**), each literal is a vertex and each sub-FA acts as an edge. This graph is then passed to the Rose subengine for deduplication of FAs, compilation into bytecode, and execution orchestration. Notably, in the subsequent (**RoseGraph**) representation, the sub-FAs are instead attached directly to their associated literal vertices via **left** or **suffix** properties. Multiple FAs may be merged into an FA with multiple initial states. Therefore, the **suffix** property also contains a sub-property **top** indicating which initial state should be chosen.

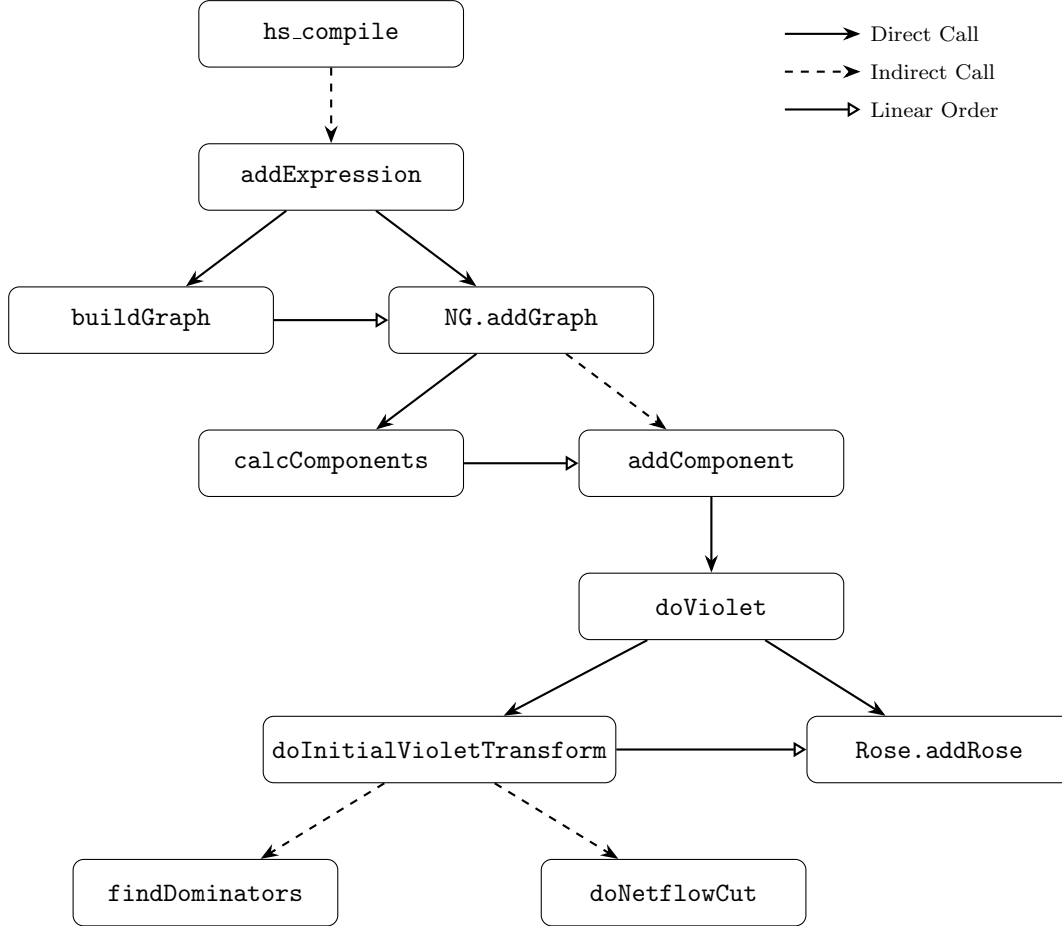


Figure 2.1: Dependency graph of main functions in the compilation phase.

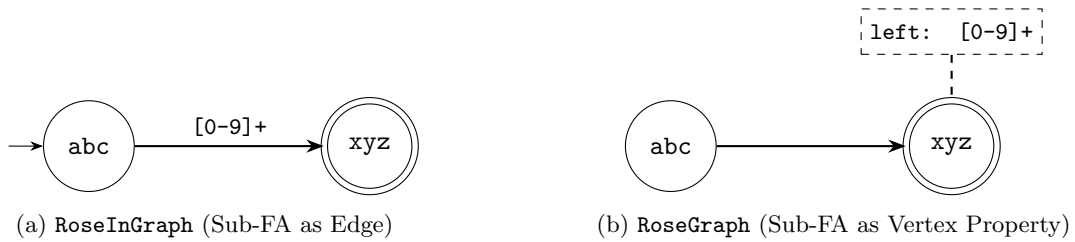


Figure 2.2: The dependency graphs by Violet and Rose.

Consider an example of a regular expression `abc[0-9]+xyz`. If we discard heuristics for short literals, then the graphs returned by Violet and Rose are as in Figures 2.2a and 2.2b, respectively. They both mean that if there is a match of `abc` that *ends* at position  $i$  (excluded) and a match of `xyz` that *starts* at position  $j$  (included), such that  $j - i > 0$ , then the subtext at  $[i, j)$  must match the sub-FA `[0-9]+`. Further optimization imposes that

$$m \leq j - i \leq M,$$

where  $m$  and  $M$  are the minimum and maximum *widths* of the sub-FA, i.e., the minimum and maximum lengths of possible strings that match the sub-FA.

## 2.2 Execution

An overview of the execution phase is illustrated in Figure 2.3. The process begins at `hs_scan`. There are three execution scenarios already determined by Rose: (1) the set of patterns are all literals (2) there are no literals that wake up FAs, and (3) literals wake up FAs. The last scenario is the most common. Its flow reaches `roseBlockExec`, where now one of its string matchers is called through `hwlmExec` together with a `roseCallback` function. The `roseCallback` function eventually calls `roseRunProgram`. Here is where the `left` or `suffix` FAs are checked, waken up and executed.

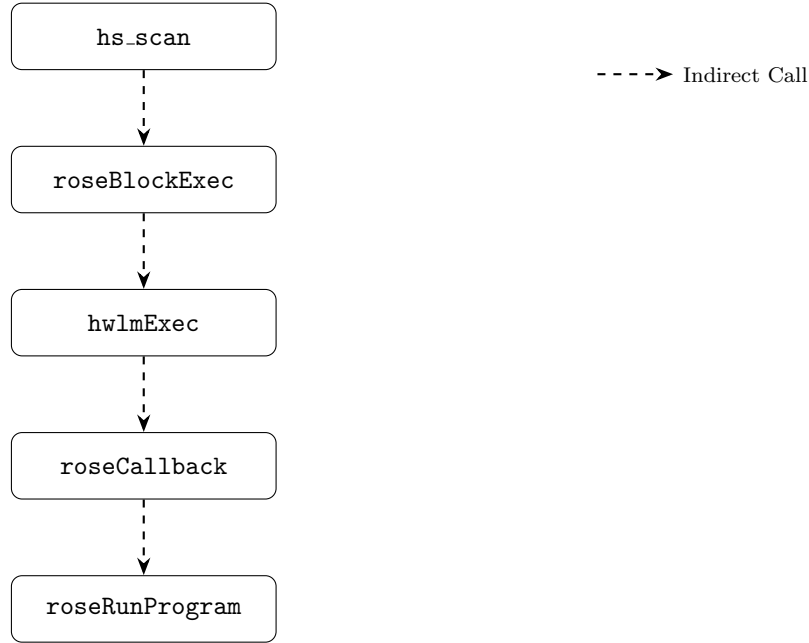


Figure 2.3: Dependency graph of main functions in the execution phase.

## 2.3 The FDR Matcher

There are at least four SIMD-based string matchers in Hyperscan: Noodle is used when there is a single literal, Teddy [6] or Harry [9] when there are 2 to 300, and FDR when there are more than 300. The last three matchers are all prefilter-based. Firstly, they group the literals into a set of *buckets*. Each bucket can be regarded as an indeterminate string (see e.g., [2])

**Definition 2.1.** Let  $\Sigma$  be an alphabet. An indeterminate string  $P$  of length  $n$  over  $\Sigma$  is a sequence of  $n$  non-empty sets of characters from  $\Sigma$ , i.e.,

$$P = p_1 p_2 \dots p_n,$$

where  $\emptyset \neq p_i \subseteq \Sigma$  for all  $i \in [1, n]$ . The subset  $\Sigma$  is usually referred to as the *don't care* character. A string  $T$  over  $\Sigma$  matches the indeterminate string  $P$  if and only if  $|T| = n$  and  $T[i] \in p_i$  for all  $i \in [1, n]$ . ■

Let us remark that the set of indeterminate strings over  $\Sigma$  is a strict subset of the set of star-free regular expressions over  $\Sigma$ . For example, the regular expression `ab|cd` is not an indeterminate string. Let

$\{P_1, P_2, \dots, P_k\}$  be the set of literals in a bucket. Let  $p_{ij}$  be the  $j$ -th character of literal  $P_i$ . The literals are right-aligned and padded with the don't care character  $*$  to form an indeterminate string  $\mathbf{P}$  of length  $n = \max\{|P_1|, |P_2|, \dots, |P_k|\}$ , i.e.,  $\mathbf{P} = p_1 p_2 \dots p_n$ , where

$$p_{n-j} = \begin{cases} \{p_{i, |P_i|-j} \mid 1 \leq i \leq k, j \leq |P_i|\}, & \text{if } p_{i, |P_i|-j} \text{ is defined for every } i, \\ \Sigma, & \text{otherwise.} \end{cases}$$

Let  $T$  be a text. The string matchers attempt to check if  $T$  matches  $\mathbf{P}$ , and if so, which literal  $P_i$  is matched. The FDR matcher is an SIMD extension of the Shift-Or algorithm (Appendix A.1).

## 2.4 Related Problems

### 2.4.1 Optimizing regular expression decomposition

Both representations in Figure 2.2 are likely suboptimal for theoretical analysis. Firstly, the sub-FA may appear several times, even though it only appear once in the regular expression, for example in the case of  $(abc|def)[0-9]^+xyz$ . Secondly, although we had to mark  $xyz$  as a terminal vertex in Figure 2.2, this is not strictly accurate: the matching process terminates at the end of the sub-FA, not at the end of  $xyz$ . If there are remaining FAs or literals to the right, it is the sub-FA for  $[0-9]^+$  that will determine the next matching position. Therefore, we propose a more convenient representation as follows. Each vertex is either a literal or a sub-FA. There are three types of directed edges.

- Edges that uses the *end* positions of the matches of the source vertex to determine the *start* positions of the matches of the target vertex. They connect two literals or a literal and a sub-FA. We call this type of edge **setStart**.
- Edges that uses the *start* positions of the matches of the source vertex to determine the *end* positions of the matches of the target vertex. They connect a literal and a sub-FA. We call this type of edge **setEnd**.
- Edges that acts as a pure trigger, without any position information. They connect two sub-FAs. We call this type of edge **trigger**.

Consider another example of  $abc[0-9]^+xyz(de)^*$ . In its dependency graph in the new representation is as in Figure 2.4.

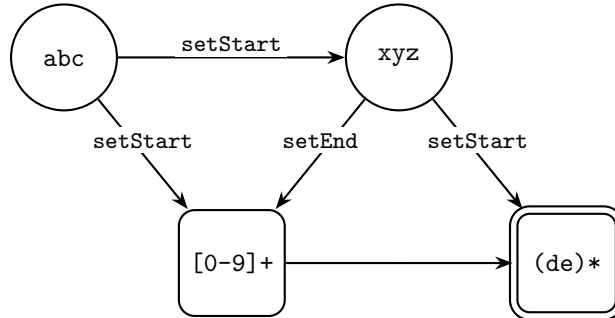


Figure 2.4: Proposed sequential dependency graph representation for  $abc[0-9]^+xyz$ .



### 2.4.2 Analysis of the FDR string matcher

### 2.4.3 Statistics of regular expressions

Suppose that the regex trees are generated according to a BST-like distribution. The average length of the longest literals of an  $n$ -node tree satisfies the recurrence

$$Y_{n+2} \stackrel{d}{=} I \left( Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right) + J \max \left( Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right), n \geq 1, \quad Y_1 = 1, Y_2 = 0. \quad (2.1)$$

Superscripts denote mutually independent copies. The variable  $U_n$  is uniformly distributed on  $\{1, \dots, n\}$ . The variables  $I$  and  $J$  are Bernoulli distributed with expectations  $u$  and  $v$  respectively, such that  $I+J \leq 1$ . We reuse the same variables  $Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)}$  and  $U_n$  across both terms, since the branches are not taken at the same time. The variables  $I$  and  $J$  are also reused, since at every new iteration, we sample a new subtree independently.

Details analysis for the asymptotic behavior of  $\mathbb{E}[Y_n]$  is given in the next chapter.

# 3

## The Length of Longest Literals

### 3.1 Lower and upper bounds

Denote  $y_n := \mathbb{E}[Y_n]$  for  $n \geq 1$ . Taking the expectation of both sides in the recurrence (2.1), we have for  $n \geq 1$ ,

$$\begin{aligned} y_{n+2} &= \frac{u}{n} \sum_{i=1}^n (y_i + y_{n+1-i}) + \frac{v}{n} \mathbb{E} \left[ \max \left( Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right] \\ &= \frac{2u}{n} \sum_{i=1}^n y_i + \frac{v}{n} \mathbb{E} \left[ \max \left( Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right]. \end{aligned}$$

Using the inequality  $\max(a, b) \leq a + b$  for  $a, b \geq 0$ , we have

$$y_{n+2} \leq \frac{2(u+v)}{n} \sum_{i=1}^n y_i.$$

Since the right-hand side of the inequality is an increasing function of  $(y_1, \dots, y_n)$ , the sequence  $(y_n)$  is bounded above by the solution to the recurrence

$$z_{n+2} = \frac{2(u+v)}{n} \sum_{i=1}^n z_i, \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (3.1)$$

Similarly, the inequality  $\max(a, b) \geq \frac{1}{2}(a+b)$  gives a lower bound sequence

$$z_{n+2} = \frac{2u+v}{n} \sum_{i=1}^n z_i, \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (3.2)$$

It is possible to employ a transfer theorem in analytic combinatorics to analyze the asymptotic behavior of these recurrences (Lemma A.1). The lower bound can be improved by tightening the lower bound of the expectation of the maximum

$$\mathbb{E} \left[ \max \left( Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right] \geq \max \left( \mathbb{E} \left[ Y_i^{(1)} \right], \mathbb{E} \left[ Y_{n+1-i}^{(2)} \right] \right) = \max(y_i, y_{n+1-i}).$$

Note that this bound is not sharp, the equality holds only when  $Y_i^{(1)} = Y_{n+1-i}^{(2)}$  a.s. The lower bound recurrence is then given by

$$z_{n+2} = \frac{2u}{n} \sum_{i=1}^n z_i + \frac{v}{n} \sum_{i=1}^n \max(z_i, z_{n+1-i}), \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (3.3)$$

Detailed analysis of (3.3) is given in the next section.

## 3.2 Asymptotics of the lower bound recurrence

The sequence defined by (3.3) is bounded between those defined by (3.1) and (3.2). If  $2u + v > 1$ , then both (3.1) and (3.2) grow as a pure power of  $n$ , without any lower-order factor like the logarithm. We conjecture that in this case (3.3) also grows as a pure power of  $n$ . In this section, we always refer to  $(z_n)$  as the sequence defined by (3.3), unless otherwise stated. The set of conditions on the parameters  $u$  and  $v$  is thus given by

$$u, v \geq 0, \quad u < 1, \quad u + v \leq 1, \quad 2u + v > 1. \quad (3.4)$$

In Theorem 3.10, we will strengthen the conditions to

$$u, v \geq 0, \quad \frac{1}{2} < u < 1, \quad u + v \leq 1. \quad (3.5)$$

Let us carry out a heuristic to find out the power. Assume that  $z_n \sim Cn^\alpha$  for some constants  $c > 0$  and  $\alpha > 0$ . Then, substituting into (3.3), we get

$$\begin{aligned} Cn^\alpha &\sim \frac{2u}{n} \sum_{i=1}^n Ci^\alpha + \frac{v}{n} \sum_{i=1}^n C \max(i^\alpha, (n+1-i)^\alpha) \\ &\sim C \left( \frac{2u}{n} \int_0^n Cx^\alpha dx + \frac{v}{n} \int_0^n C \max(x^\alpha, (n-x)^\alpha) dx \right) \\ &= C \left( \frac{2u}{n} \int_0^n x^\alpha dx + \frac{2v}{n} \cdot \int_{n/2}^n x^\alpha dx \right) \\ &= C \left( \frac{2u}{n} \cdot \frac{n^{\alpha+1}}{\alpha+1} + \frac{2v}{n} \cdot \frac{n^{\alpha+1} - (n/2)^{\alpha+1}}{\alpha+1} \right) \\ &= \frac{C}{\alpha+1} (2(u+v) - v2^{-\alpha}) n^\alpha. \end{aligned}$$

Thus, we must have

$$\alpha + v2^{-\alpha} = 2(u+v) - 1. \quad (3.6)$$

Note that  $\alpha > 0$ , since otherwise  $2u+v = 1$ . A natural goal is to try to escape from the maximum operator. However,  $(z_n)$  fluctuates in the first iterations, and the time where it begins to increase depends on the parameters  $u$  and  $v$ . An extreme case is that when

$$u \rightarrow 1/2 + \text{ and } v \rightarrow 0+,$$

the sequence  $(z_n)$  fluctuates for a long time before increasing consistently. It turns out that this property of *eventual monotonicity* is sufficient to escape from the maximum operator. Indeed, assume that there exists  $N_1$  such that  $z_{n+1} \geq z_n$  for all  $n \geq N_1$ . Since  $(z_n)$  is unbounded, there exists  $N_2 \geq N_1$  such that for all  $n \geq N_2$ , we have both  $z_{n+1} \geq \max_{1 \leq i \leq N_1} z_i$  and  $z_{n+1} \geq z_n$ . That is

$$\forall n \geq N_2, \quad z_{n+1} \geq \max_{1 \leq i \leq n} z_i.$$

Taking  $N_3 = 2N_2 - 1$ , we have

$$\forall n \geq N_3, \forall \frac{N_3+1}{2} \leq i \leq N_3, \quad z_i \geq z_{n+1-i}.$$

The question about when it stops fluctuating is itself interesting, which we leave open.

**Open question 1** *In terms of  $u$  and  $v$ , what is the first index  $N$  such that  $(z_n)_{n \geq N}$  is increasing?*

### 3.2 Asymptotics of the lower bound recurrence

---

We start with weaker results that do not require escaping from the maximum operator. Define

$$F(x, y) = u(x + y) + v \max(x, y), x, y \in [0, 1]. \quad (3.7)$$

We rewrite our recurrence as

$$z_{n+2} = \frac{1}{n} \sum_{i=1}^n F(z_i, z_{n+1-i}), \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (3.8)$$

If  $z_n \sim Cn^\alpha$ , we can try to approximate  $F$  as follows.

**Lemma 3.1** *For every  $\alpha \in (0, 1]$ , we have*

$$\frac{1}{n} \sum_{i=1}^n F(i^\alpha, (n+1-i)^\alpha) \sim n^\alpha \int_0^1 F(x^\alpha, (1-x)^\alpha).$$

*Proof.* Rewrite the asymptotics as a limit

$$\frac{1}{n} \sum_{i=1}^n F\left(\left(\frac{i}{n}\right)^\alpha, \left(1 - \frac{i}{n} + \frac{1}{n}\right)^\alpha\right) \rightarrow \int_0^1 F(x^\alpha, (1-x)^\alpha).$$

The function on the right-hand side is continuous on  $[0, 1]$ , and the left-hand side is different from its Riemann sum only by a perturbation  $1/n$  in the second argument. We show that the perturbation is negligible. Define for  $x \in [0, 1]$  the functions

$$f_n(x) = F\left(x^\alpha, \left(1 - x + \frac{1}{n}\right)^\alpha\right) \text{ and } f(x) = F(x^\alpha, (1-x)^\alpha),$$

For  $a, b, c \geq 0$  and  $\alpha \in (0, 1]$ , we have

$$|\max(a, b) - \max(a, c)| \leq |b - c| \text{ and } (a + b)^\alpha \leq a^\alpha + b^\alpha.$$

Hence,

$$\|f_n - f\|_\infty \leq \sup_{x \in [0, 1]} (u + v) \left( \left(1 - x + \frac{1}{n}\right)^\alpha - (1 - x)^\alpha \right) \leq (u + v) \frac{1}{n^\alpha} \rightarrow 0.$$

Therefore,

$$\left| \frac{1}{n} \sum_{i=1}^n f_n\left(\frac{i}{n}\right) - \frac{1}{n} \sum_{i=1}^n f\left(\frac{i}{n}\right) \right| \leq \|f_n - f\|_\infty \rightarrow 0.$$

□

Let

$$\Lambda(\alpha) = \int_0^1 F(x^\alpha, (1-x)^\alpha) dx = \frac{2(u+v) - v2^{-\alpha}}{\alpha + 1}.$$

Then  $\Lambda$  is strictly decreasing. Indeed,

$$\begin{aligned} \Lambda'(\alpha) &= \frac{v2^{-\alpha} \ln 2}{\alpha + 1} - \frac{2(u+v) - v2^{-\alpha}}{(\alpha + 1)^2} = \frac{v2^{-\alpha}((\alpha + 1) \ln 2 + 1) - 2(u+v)}{(\alpha + 1)^2}, \\ (2^{-\alpha}((\alpha + 1) \ln 2 + 1))' &= -2^{-\alpha}(\alpha + 1) \ln^2(2) < 0. \end{aligned}$$

Hence,

$$(\alpha + 1)^2 \Lambda'(\alpha) < (\alpha + 1)^2 \Lambda'(0) = -(2u + (1 - \ln 2)v) < 0.$$

Therefore, there exists a unique  $\alpha$  such that  $\Lambda(\alpha) = 1$ , which is the solution to (3.6). Furthermore,  $\Lambda(\beta) < 1$  for all  $\beta > \alpha$  and  $\Lambda(\beta) > 1$  for all  $\beta < \alpha$ . That helps us prove a weaker asymptotic theorem.

### 3.2 Asymptotics of the lower bound recurrence

---

**Theorem 3.2** *Let  $\alpha$  be the solution to (3.6). Then,  $z_n = n^{\alpha+o(1)}$ .*

*Proof.* Let  $A_n(\beta) = \frac{1}{n} \sum_{i=1}^n F(i^\beta, (n+1-i)^\beta)$ . Firstly, we prove the upper bound. Assume that  $\beta > \alpha$ . Then,  $\Lambda(\beta) < 1$ . Hence there exists  $N$  such that for all  $n \geq N$ ,

$$A_n(\beta) < n^\beta < (n+2)^\beta.$$

Let  $M_n = \max_{1 \leq k \leq n} \frac{z_k}{k^\beta}$ . For  $n \geq N$ , we have

$$z_{n+2} = \frac{1}{n} \sum_{i=1}^n F(z_i, z_{n+1-i}) \leq M_n A_n(\beta) < M_n (n+2)^\beta.$$

Hence,  $\frac{z_{n+2}}{(n+2)^\beta} < M_n$ . Or  $(M_n)$  is eventually non-increasing. Therefore,  $z_n = O(n^\beta)$ .

We argue similarly for the lower bound, with an adjustment for  $z_2$ . We have

$$A_n(\beta) \sim n^\beta \sim (n+2)^\beta \text{ and } A_n(\beta) \sim \frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n+1-i)^\beta).$$

Assume that  $\beta < \alpha$ . Then,  $\Lambda(\beta) > 1$ . Hence there exists  $N \geq 3$  such that for all  $n \geq N$ ,

$$\frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n+1-i)^\beta) > (n+2)^\beta.$$

Let  $m_n = \min_{3 \leq i \leq n-2} \frac{z_i}{i^\beta}$ . For  $n \geq N$ , we have

$$z_{n+2} \geq \frac{1}{n} \sum_{i=3}^{n-2} F(z_i, z_{n+1-i}) \geq m_n \frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n+1-i)^\beta) > m_n (n+2)^\beta.$$

Hence,  $\frac{z_{n+2}}{(n+2)^\beta} > m_n$ . Or  $(m_n)$  is eventually increasing. Therefore,  $z_n = \Omega(n^\beta)$ . □

Next, we present the stronger big- $\Theta$  result. The proof is done by fine-grained induction. We provide integral bounds for the sum of the maximums to be used in the induction step.

**Lemma 3.3** *Let  $f : [0, \infty) \rightarrow \mathbb{R}$  be a increasing function. For every integer  $n \geq 1$ , we have*

$$S_1 := \sum_{i=0}^{n-1} \max(f(i), f(n-1-i)) \leq 2 \int_{n/2}^n f(x) dx \leq S_2 := \sum_{i=1}^n \max(f(i), f(n+1-i)).$$

*Proof.* We first prove the upper bound on the first sum. We will use the following bounds for  $t \geq 0$

$$f(t) \leq \int_t^{t+1} f(x) dx \leq 2 \int_{t+1/2}^{t+1} f(x) dx.$$

If  $n = 2k$ , we have

$$S_1 = 2 \sum_{i=k}^{2k-1} f(i) \leq 2 \sum_{i=k}^{2k-1} \int_i^{i+1} f(x) dx.$$

If  $n = 2k+1$ , we have

$$S_1 = 2 \sum_{i=k+1}^{2k} f(i) + f(k) \leq 2 \sum_{i=k+1}^{2k} \int_i^{i+1} f(x) dx + 2 \int_{k+1/2}^{k+1} f(x) dx.$$

### 3.2 Asymptotics of the lower bound recurrence

---

The omit the proof for the lower bound on the second sum, while pointing out the bounds that would be used. For  $t \geq 1$ , we have

$$f(t) \geq \int_{t-1}^t f(x) dx \geq 2 \int_{t-1}^{t-1/2} f(x) dx.$$

□

**Theorem 3.4** *Let  $\alpha$  be the solution to (3.6). Then,  $z_n = \Theta(n^\alpha)$ .*

*Proof.* We prove the lower bound. It is clear that  $z_n > 0$  for all  $n \neq 2$ . The main idea is to isolate  $z_2$ . The induction hypothesis is  $z_n \geq K(n+L)^\alpha$  for  $n \neq 2$ . The base cases and the exact constants are determined later. For  $n \geq 3$ , we have

$$\begin{aligned} nz_{n+2} &= 2u \sum_{i=1}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}) \\ &\geq K \left( 2u \sum_{i=1}^n (i+L)^\alpha + v \sum_{i=1}^n \max((i+L)^\alpha, (n+1-i+L)^\alpha) \right) - K2u(2+L)^\alpha \\ &\geq K \left( 2u \int_0^n (x+L)^\alpha dx + 2v \int_{n/2}^n (x+L)^\alpha dx \right) - K2u(2+L)^\alpha. \end{aligned}$$

Let

$$\begin{aligned} A_n &= 2u \int_0^n (x+L)^\alpha dx + 2v \int_{n/2}^n (x+L)^\alpha dx \\ &= \frac{1}{\alpha+1} \left( (2u+2v)(n+L)^{\alpha+1} - 2v \left( \frac{n}{2} + L \right)^{\alpha+1} \right). \end{aligned}$$

Using the expansions

$$\begin{aligned} (n+L)^{\alpha+1} &= n^{\alpha+1} + (\alpha+1)Ln^\alpha + O(n^{\alpha-1}), \\ \left( \frac{n}{2} + L \right)^{\alpha+1} &= 2^{-\alpha+1}n^{\alpha+1} + (\alpha+1)L2^{-\alpha}n^\alpha + O(n^{\alpha-1}), \end{aligned}$$

we get

$$\begin{aligned} A_n &= n^{\alpha+1} + L(2u+2v-2v2^{-\alpha})n^\alpha + O(n^{\alpha-1}) \\ &= n^{\alpha+1} + L(\alpha+1-v2^{-\alpha})n^\alpha + O(n^{\alpha-1}). \end{aligned}$$

On the other hand,

$$n(n+L+2)^\alpha = n^{\alpha+1} + \alpha(L+2)n^\alpha + O(n^{\alpha-1}).$$

Hence,

$$A_n - 2u(2+L)^\alpha - n(n+L+2)^\alpha = (L(1-v2^{-\alpha}) - 2\alpha)n^\alpha + O(n^{\alpha-1}).$$

Since  $1-v2^{-\alpha} > 0$ , choose  $L > \frac{2\alpha}{1-v2^{-\alpha}}$ , then there exists  $N$  such that for all  $n \geq N$ ,

$$A_n - 2u(2+L)^\alpha - n(n+L+2)^\alpha \geq 0,$$

implying that  $z_{n+2} \geq K(n+L+2)^\alpha$ . Finally, we choose  $K = \min_{i \in \{1,3,\dots,N+1\}} \frac{z_i}{(i+L)^\alpha} > 0$  to satisfy the base cases.

Now we prove the upper bound. Our induction hypothesis is  $z_n \leq K(n-1)^\alpha$  for  $n \geq 2$ , for some constant

### 3.2 Asymptotics of the lower bound recurrence

---

$K > 1$ . The base case  $z_2 = 0$  is trivially satisfied. For  $n \geq 2$ , we have

$$\begin{aligned}
nz_{n+2} &= 2u + 2u \sum_{i=2}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}) \\
&\leq K \left( 2u \sum_{i=1}^{n-1} i^\alpha + v \sum_{i=0}^{n-1} \max(i^\alpha, (n-1-i)^\alpha) \right) + 2u \\
&\leq K \left( 2u \int_1^n x^\alpha dx + 2v \int_{n/2}^n x^\alpha dx \right) + 2u \\
&= K \left( 2u \frac{n^{\alpha+1} - 1}{\alpha + 1} + 2v \frac{n^{\alpha+1} - (n/2)^{\alpha+1}}{\alpha + 1} \right) + 2u \\
&= Kn^{\alpha+1} \frac{2u + 2v - v2^{-\alpha}}{\alpha + 1} - \frac{2Ku}{\alpha + 1} + 2u \\
&= Kn^{\alpha+1} - \frac{2Ku}{\alpha + 1} + 2u.
\end{aligned}$$

The first inequality is valid by noticing that

$$\max(z_1, z_n) = \max(1, z_n) \leq \max(1, K(n-1)^\alpha) = \max(0, K(n-1)^\alpha).$$

The second inequality uses Lemma 3.3. Choose  $K = \alpha + 1 > 1$ , then  $z_{n+2} \leq Kn^\alpha \leq K(n+2)^\alpha$ .  $\square$

**Remark 3.1.** In the proof of the lower bound we exploit the fact that  $(n+L)^{\alpha+1}$  dominates  $n(n+L+2)^{\alpha+1}$  when  $L$  is large enough. Actually, we also have  $(n-L)^{\alpha+1} \ll n(n-L+2)^{\alpha+1}$  when  $L$  is large enough. However, if we apply the same idea to the upper bound, we cannot start the hypothesis from the beginning, but only after some  $n > L$ . It is nice that we can overcome that by choosing  $L = 1$  and saving a fixed budget  $2u$  in the first term.

We proceed to the exact asymptotics  $z_n \sim Cn^\alpha$ . The idea is to use loose inequalities to prove easier quantitative properties, then upgrade step by step with tighter inequalities.

**Lemma 3.5 (Vanishing difference)** *We have  $|z_{n+1} - z_n| \rightarrow 0$ .*

*Proof.* For  $n \geq 2$ , we have

$$\begin{aligned}
nz_{n+2} &= 2u \sum_{i=1}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}), \\
(n-1)z_{n+1} &= 2u \sum_{i=1}^{n-1} z_i + v \sum_{i=1}^{n-1} \max(z_i, z_{n-i}).
\end{aligned}$$

Hence,

$$\begin{aligned}
n(z_{n+2} - z_{n+1}) &= 2uz_n + v \max(z_n, z_1) - z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_i, z_{n+1-i}) - \max(z_i, z_{n-i})) \\
&= 2uz_n + v \max(z_n, z_1) - z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)).
\end{aligned} \tag{3.9}$$

Using  $\max(z_n, z_1) \leq z_n + z_1$ , we have

$$\begin{aligned}
n(z_{n+2} - z_{n+1}) &\leq 2uz_n + v(z_n + z_1) - v z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)) \\
&\leq 2uz_n + v z_1 + v(z_n - z_{n+1}) + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i))
\end{aligned}$$

### 3.2 Asymptotics of the lower bound recurrence

Using  $|\max(a, b) - \max(a, c)| \leq |b - c|$ , we have

$$\begin{aligned} n|z_{n+2} - z_{n+1}| &\leq 2uz_n + vz_1 + v|z_{n+1} - z_n| + v \sum_{i=1}^{n-1} |z_{i+1} - z_i| = 2uz_n + vz_1 + v \sum_{i=1}^n |z_{i+1} - z_i| \\ &\leq 2uz_n + vz_1 + nv \sup_{1 \leq i \leq n} |z_{i+1} - z_i|. \end{aligned}$$

Since  $u < 1$ , we have  $\alpha < 1$ . By  $z_n = \Theta(n^\alpha)$ , we have  $\frac{1}{n}(2uz_n + vz_1) \rightarrow 0$ , or

$$|z_{n+2} - z_{n+1}| \leq v \sup_{1 \leq i \leq n} |z_{i+1} - z_i| + o(1).$$

We have  $v < 1$  since otherwise,  $2u + v > 1$  would be violated. Hence  $(|z_{i+1} - z_i|)$  is bounded. Let  $M = \sup_{n \geq 1} |z_{n+1} - z_n|$ . Let  $\epsilon > 0$ , then there exists  $N$  such that for all  $n \geq N$ ,  $|z_{n+1} - z_n| < M + \epsilon$ . Therefore,

$$|z_{n+2} - z_{n+1}| \leq v(M + \epsilon) + o(1).$$

Let  $n \rightarrow \infty$ , we have  $M \leq v(M + \epsilon)$ . Since  $\epsilon > 0$  is arbitrary, we have  $M = 0$ .  $\square$

Since  $(z_n)$  is unbounded, we have the following corollary.

**Corollary 3.6 (Eventual exceedance)** *Let  $\alpha$  be the solution to (3.6). Then, for every  $M > 0$ , there exists  $N$  such that for all  $n \geq N$ ,  $z_n > M$ .*

**Lemma 3.7 (Increment identity)** *For  $n \geq 1$ , let  $\Delta_n = z_{n+1} - z_n$ . Then for all sufficiently large  $n$ , there exist numbers  $\theta_{n,i} \in [0, 1]$  for  $1 \leq i \leq n-1$  such that*

$$n\Delta_{n+1} = (2u + v - 1)z_n - \Delta_n + v \sum_{i=1}^{n-1} \theta_{n,i} \Delta_i. \quad (3.10)$$

*Proof.* We start from (3.9)

$$n\Delta_{n+1} = 2uz_n + v \max(z_n, z_1) - z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_i, z_{n+1-i}) - \max(z_i, z_{n-i})).$$

Now with Corollary 3.6, we have  $\max(z_n, z_1) = z_n$  for all sufficiently large  $n$ . Hence, for all sufficiently large  $n$ ,

$$n\Delta_{n+1} = (2u + v - 1)z_n - \Delta_n + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)).$$

For every  $a \geq 0$ , the function  $x \mapsto \max(a, x)$  is increasing and 1-Lipschitz. Hence for each  $n, i$  there is some  $\theta_{n,i} \in [0, 1]$  satisfying (3.10).  $\square$

To this step, we have isolated the increment sequence  $(\Delta_n)$  from the original sequence  $(z_n)$ . Our next goal is to prove that  $(\Delta_n)$  is *eventually positive*. That would be done if in (3.10), we can absorb a lower bound of  $-\Delta_n + v \sum_{i=1}^{n-1} \theta_{n,i} \Delta_i$  into  $(2u + v - 1)z_n$ . We already have  $|\Delta_n| = o(z_n)$  since  $(\Delta_n)$  is bounded while  $z_n \rightarrow \infty$ . Because we do not know much about  $\theta_{n,i}$ , we can only apply the worst case to them. A basic inequality for absolute value leads to showing that  $\sum_{i=1}^n |\Delta_i| = o(z_n)$ . Unfortunately, if it is true that  $z_n \sim Cn^\alpha$  and  $z_n$  is eventually increasing, then  $\sum_{i=1}^n |\Delta_i| \sim Cn^\alpha$ . Therefore, we need a tighter lower bound. We will use the *negative variation* of  $\Delta_n$ . For every  $x \in \mathbb{R}$ , we have

$$x_- := \max(-x, 0) = \frac{1}{2}(|x| - x) \geq -x \iff x \geq -x_-. \quad (3.11)$$

Define the negative variation series  $(\Delta_n)$  as

$$V_n := \sum_{i=1}^n (\Delta_i)_-. \quad (3.12)$$

Then  $(V_n)$  is increasing. We state the criterion for eventual increase of  $(z_n)$  in terms of  $(V_n)$  as follows.



### 3.2 Asymptotics of the lower bound recurrence

**Lemma 3.8 (Negative variation criterion)** *If  $V_n = o(z_n)$ , then  $(z_n)$  is eventually increasing.*

*Proof.* By Lemma 3.7, for all sufficiently large  $n$ ,

$$n\Delta_{n+1} \geq (2u + v - 1)z_n - |\Delta_n| - vV_n.$$

Since  $\Delta_n$  is bounded and  $z_n \rightarrow \infty$ , we have  $|\Delta_n| = o(z_n)$ . By assumption,  $V_n = o(z_n)$ . Hence for all sufficiently large  $n$ ,

$$n\Delta_{n+1} \geq \frac{1}{2}(2u + v - 1)z_n > 0.$$

□

It would be perfect if we could prove that  $(V_n)$  is bounded, as this is indeed the truth. However, it is already good to prove that  $V_n = o(n^\alpha)$ , since we already have  $z_n = \Theta(n^\alpha)$ .

**Lemma 3.9** *We have  $V_n = O(n^v)$ .*

*Proof.* Again, we start from (3.10). There exists  $N$  such that for all  $n \geq N$ ,

$$n\Delta_{n+1} = (2u + v - 1)z_n - \Delta_n + v \sum_{i=1}^{n-1} \theta_{n,j} \Delta_i.$$

Let  $M = \sup |\Delta_n|$ . If  $\Delta_{n+1} < 0$ , then

$$\begin{aligned} n(\Delta_{n+1})_- &= -n\Delta_{n+1} = |\Delta_n| - (2u + v - 1)z_n - v \sum_{j=1}^{n-1} (\Delta_j)_- \\ &\leq vV_{n-1} + |\Delta_n| - (2u + v - 1)z_n \\ &\leq vV_{n-1} + M. \end{aligned}$$

Hence,  $(\Delta_{n+1})_- \leq \frac{v}{n}V_{n-1} + \frac{M}{n} \leq \frac{v}{n}V_n + \frac{M}{n}$ . Therefore,

$$V_{n+1} \leq \left(\frac{v}{n} + 1\right) V_n + \frac{M}{n}. \quad (3.13)$$

Note that if  $\Delta_{n+1} \geq 0$ , then  $(\Delta_{n+1})_- = 0$ , leading to  $V_{n+1} = V_n$ , or (3.13) holds automatically. We then have

$$V_{n+1} + \frac{M}{v} \leq \left(\frac{v}{n} + 1\right) \left(V_n + \frac{M}{v}\right) \leq \prod_{i=N}^n \left(\frac{v}{i} + 1\right) \left(V_N + \frac{M}{v}\right).$$

We are left to estimate the product. Taking the logarithm gives

$$\begin{aligned} \ln \left( \prod_{i=N}^n \left(1 + \frac{v}{i}\right) \right) &= \sum_{i=N}^n \ln \left(1 + \frac{v}{i}\right) \\ &\leq \sum_{i=N}^n \frac{v}{i} = v(H_n - H_{N-1}) \\ &\leq vH_n \leq v \ln n + C, \end{aligned}$$

where  $H_n = \sum_{i=1}^n \frac{1}{i} = \ln n + \gamma + O(1/n)$  is the  $n$ -th harmonic number and  $C$  is some constant. Taking back the exponential of both sides yields

$$\prod_{i=N}^n \left(1 + \frac{v}{i}\right) \leq e^C n^v = O(n^v).$$

□

### 3.2 Asymptotics of the lower bound recurrence

---

We state our checkpoint as a theorem.

**Theorem 3.10** *If  $v < \alpha$ , then  $(z_n)$  is eventually increasing.*

Let us reduce the condition  $v < \alpha$  in terms of  $u$  and  $v$ . Define

$$f(\alpha) = \alpha + 1 - 2(u + v) + v2^{-\alpha}, \alpha \in [0, 1].$$

Then  $f'(\alpha) = 1 - v2^{-\alpha} \ln 2 > 1 - \ln 2 > 0$  for all  $\alpha > 0$ . Hence  $v < \alpha$  is equivalent to  $f(v) < 0$  i.e.,

$$v + 1 - 2(u + v) + v2^{-v} < 0 \iff u > \frac{1}{2}(1 - v + v2^{-v}).$$

If we impose conditions 3.5, then the above inequality is satisfied. The case  $v \geq \alpha$  is left open. With  $(z_n)$  being eventually increasing, it has a nicer recurrence without the whole history.

**Lemma 3.11** *There exists  $N \geq 1$  such that the sequence  $(z_n)_{n \geq N}$  defined by 3.3 satisfies the recurrence*

$$nz_{n+2} - (n-1)z_{n+1} = 2(u+v)z_n - vz_{\lceil (n-1)/2 \rceil}. \quad (3.14)$$

with  $z_{N+1} \geq z_N \geq \dots \geq z_{\lceil (N-1)/2 \rceil} \geq 1$ .

*Proof.* Since  $(z_n)$  is eventually increasing, there exists  $N$  such that for all  $n \geq N$ , we have  $z_n \geq z_i$  for all  $1 \leq i \leq n-1$  and  $z_n \geq 1$ . Hence, for all  $n \geq N+1$ ,

$$\begin{aligned} nz_{n+2} &= 2u \sum_{i=1}^n z_i + 2v \sum_{i=\lceil n/2 \rceil+1}^n z_i + \delta_n vz_{\lceil n/2 \rceil} \\ (n-1)z_{n+1} &= 2u \sum_{i=1}^{n-1} z_i + 2v \sum_{i=\lceil (n-1)/2 \rceil+1}^{n-1} z_i + \delta_{n-1} vz_{\lceil (n-1)/2 \rceil}, \end{aligned}$$

where  $\delta_n = 1$  if  $n$  is odd and 0 if  $n$  is even. If  $n = 2k$ , then  $\lceil n/2 \rceil = \lceil (n-1)/2 \rceil = k$ ,  $\delta_n = 0$  and  $\delta_{n-1} = 1$ . Hence,

$$2kz_{2k+2} - (2k-1)z_{2k+1} = 2(u+v)z_{2k} - vz_k.$$

If  $n = 2k+1$ , then  $\lceil n/2 \rceil = \lceil (n-1)/2 \rceil = k$ ,  $\delta_n = 1$  and  $\delta_{n-1} = 0$ . Hence,

$$(2k+1)z_{2k+3} - (2k)z_{2k+2} = 2(u+v)z_{2k+1} - 2vz_{k+1} + vz_{k+1} = (2k+1)(u+v)z_{2k+1} - vz_{k+1}.$$

In both cases, we arrive at (3.14) □

**Theorem 3.12** *Let  $(z_n)$  be the defined by (3.14) and  $\alpha$  be the solution to (3.6). Then  $z_n \sim Cn^\alpha$  for some constant  $C > 0$ .*

*Proof.* Let  $x_n = \frac{z_n}{(n-2)^\alpha}$  for  $n \geq N$ . We will prove that  $(x_n)$  converges by proving that  $\sum_{n \geq 0} |x_{n+1} - x_n|$  converges. We have, for  $n \geq 5$ ,

$$\begin{aligned} n^{\alpha+1}x_{n+1} - (n-1)^{\alpha+1}x_n &= 2(u+v)(n-2)^\alpha x_{n-1} - v \left( \left\lfloor \frac{n-1}{2} \right\rfloor - 1 \right)^\alpha x_{\lfloor (n-1)/2 \rfloor} \\ \iff x_{n+1} &= \left( 1 - \frac{1}{n} \right)^{\alpha+1} x_n + \frac{2(u+v)}{n} \left( 1 - \frac{2}{n} \right)^\alpha x_{n-1} - \frac{v}{n} \left( \frac{\lfloor (n-1)/2 \rfloor - 1}{n} \right)^\alpha x_{\lfloor (n-1)/2 \rfloor}. \end{aligned}$$

We have

$$\begin{aligned} \left( 1 - \frac{1}{n} \right)^{\alpha+1} &= 1 - \frac{\alpha+1}{n} + O\left( \frac{1}{n^2} \right), \\ \frac{1}{n} \left( 1 - \frac{2}{n} \right)^\alpha &= \frac{1}{n} + O\left( \frac{1}{n^2} \right), \\ -\frac{1}{n} \left( \frac{\lfloor (n-1)/2 \rfloor - 1}{n} \right)^\alpha &= -\frac{1}{n} \left( 2^{-\alpha} + O\left( \frac{1}{n} \right) \right) = -\frac{2^{-\alpha}}{n} + O\left( \frac{1}{n^2} \right). \end{aligned}$$

### 3.2 Asymptotics of the lower bound recurrence

---

By the previous lemma,  $(z_n)$  is bounded. Therefore,

$$\begin{aligned} z_{n+1} - z_n &= -\frac{\alpha+1}{n}z_n + \frac{2(u+v)}{n}z_{n-1} - \frac{v2^{-\alpha}}{n}z_{\lfloor (n-1)/2 \rfloor} + O\left(\frac{1}{n^2}\right) \\ &= -\frac{2(u+v)}{n}(z_n - z_{n-1}) + \frac{v2^{-\alpha}}{n}(z_n - z_{\lfloor (n-1)/2 \rfloor}) + O\left(\frac{1}{n^2}\right). \end{aligned}$$

Let  $\nabla z_n = z_n - z_{n-1}$ . Then, there exists a constant  $L > 0$  such that for every  $n \geq 5$ ,

$$|\nabla z_{n+1}| \leq \frac{2(u+v)}{n}|\nabla z_n| + \frac{v2^{-\alpha}}{n} \sum_{i=\lfloor (n+1)/2 \rfloor}^n |\nabla z_i| + \frac{L}{(n+1)^2}.$$

Equivalently, for every  $n \geq 6$ ,

$$|\nabla z_n| \leq \frac{2(u+v)}{n-1}|\nabla z_{n-1}| + \frac{v2^{-\alpha}}{n-1} \sum_{i=\lfloor n/2 \rfloor}^{n-1} |\nabla z_i| + \frac{L}{n^2}.$$

Let  $u_m = \max_{2^m \leq n < 2^{m+1}} |\nabla z_n|$ . Then, for every  $n \in [2^m, 2^{m+1})$ , we have

$$\begin{aligned} \frac{1}{n-1}|\nabla z_{n-1}| &\leq \frac{1}{n-1} \max_{2^{m-1} \leq i < 2^m} |\nabla z_i| = \frac{1}{n-1} \max(u_{m-1}, u_m), \\ \frac{1}{n-1} \sum_{i=\lfloor n/2 \rfloor}^{n-1} |\nabla z_i| &\leq \frac{n+1}{2(n-1)} \max_{2^{m-1} \leq i < 2^m} |\nabla z_i| \leq \left(\frac{1}{2} + \frac{1}{n-1}\right) \max(u_{m-1}, u_m). \end{aligned}$$

Therefore,

$$\begin{aligned} |\nabla z_n| &\leq \frac{2(u+v)}{n-1} \max(u_{m-1}, u_m) + v2^{-\alpha} \left(\frac{1}{2} + \frac{1}{n-1}\right) \max(u_{m-1}, u_m) + \frac{L}{2^{2m}} \\ &= \left(\frac{\alpha+1}{n-1} + v2^{-\alpha-1}\right) \max(u_{m-1}, u_m) + \frac{L}{2^{2m}}. \end{aligned}$$

Since  $v2^{-\alpha} = 2(u+v) - \alpha - 1 < 1$ , we can choose  $m_0$  large enough such that for every  $m \geq m_0$  and  $n \in [2^m, 2^{m+1})$ , there exists  $\gamma_1$  such that  $\frac{\alpha+1}{n-1} + v2^{-\alpha-1} < \gamma_1 < \frac{1}{2}$ . Taking the maximum over  $n \in [2^m, 2^{m+1})$ , we have

$$u_m \leq \gamma_1 \max(u_{m-1}, u_m) + \frac{L}{2^{2m}}.$$

If  $u_{m-1} > u_m$ , then  $u_m \leq \gamma_1 u_{m-1} + \frac{L}{2^{2m}}$ .

If  $u_{m-1} \leq u_m$ , then  $u_m \leq \frac{L}{2^{2m}(1-\gamma_1)}$ .

In both cases, we have

$$\begin{aligned} u_m &\leq \gamma_1 u_{m-1} + \frac{L}{2^{2m}(1-\gamma_1)} \\ &\leq \sum_{i=m_0+1}^m \gamma_1^{m-i} \frac{L}{2^{2i}(1-\gamma_1)} + \gamma_1^{m-m_0} u_{m_0} \\ &< \frac{L}{(1-\gamma_1)} \sum_{i=m_0+1}^m \gamma_1^{m-i} \left(\frac{1}{4}\right)^i + \gamma_1^{m-m_0} u_{m_0}. \end{aligned}$$

### 3.3 Asymptotics

---

Choose  $\gamma_2$  such that  $\max\left(\gamma_1, \frac{1}{4}\right) < \gamma_2 < \frac{1}{2}$ . We have

$$u_m < \frac{L}{(1-\gamma_1)} \sum_{i=m_0+1}^m \gamma_2^m + \gamma_2^{m-m_0} u_{m_0} \leq L' m \gamma_2^m,$$

for some constant  $L' > 0$ . Choose  $\gamma_3$  such that  $\gamma_2 < \gamma_3 < \frac{1}{2}$ . Then there exists  $m_1 > m_0$  such that for every  $m \geq m_1$ , we have  $L' m \gamma_2^m < L' \gamma_3^m$ , implying that,

$$u_m < L' \gamma_3^m.$$

Therefore,

$$\sum_{n=0}^{\infty} |\nabla z_n| = \sum_{m=0}^{\infty} \sum_{n=2^m}^{2^{m+1}-1} |\nabla z_n| = \sum_{m=0}^{\infty} 2^m u_m < \sum_{m=0}^{m_1-1} 2^m u_m + L' \sum_{m=m_1}^{\infty} (2\gamma_3)^m < \infty.$$

Hence,  $(z_n)$  converges to some constant  $C > 0$ . Thus,  $x_n \sim Cn^\alpha$ . □

### 3.3 Asymptotics

With the results that the lower and upper bounds grow as big- $\Theta$  of a pure power of  $n$ , it is plausible to conjecture the same situation for the original sequence  $(y_n)$ . We adapt the contraction method (e.g, [5]). The original idea is as follows.

1. Guess the scaling factor, then derive a normalized recurrence.
2. Derive a limiting fixed-point equation from the normalized recurrence.
3. Prove that the limiting equation has a unique solution in a suitable metric space using a contraction argument.
4. Prove that the normalized sequence converges in distribution to that unique solution.

We discuss the reality that the contraction method is *not* directly applicable to our case. Let  $X_n = \frac{Y_n}{n^\alpha}$  for some  $\alpha > 0$  to be determined. We have the recurrence for  $X_n$

$$\begin{aligned} X_{n+2} \stackrel{d}{=} I \left( \left( \frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)} + \left( \frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \\ + J \max \left( \left( \frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)}, \left( \frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \end{aligned} \quad (3.15)$$

The idea is to derive a limiting equation, then prove that the scaled sequence  $(X_n)$  converges in distribution to a unique solution of the limiting equation. In our case, we state the limiting equation as follows.

**Lemma 3.13** *Suppose that  $X_n \xrightarrow{d} X$ . Then,  $X$  satisfies the following distributional equation*

$$X \stackrel{d}{=} I \left( U^\alpha X^{(1)} + (1-U)^\alpha X^{(2)} \right) + J \max(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)}), \quad (3.16)$$

where  $U \sim \text{Unif}[0, 1]$ .

### 3.3 Asymptotics

---

The proof of Lemma 3.13 is given in Appendix A.4.1. Our first goal is to prove that there exists a unique solution  $(X, \alpha)$  to (3.16). We define the operator induced by the right-hand side of (3.16) by  $T : \mathcal{D} \rightarrow \mathcal{D}$  such that

$$T_\alpha(\mu) = \mathcal{L}\left(I\left(U^\alpha X^{(1)} + (1 - U)^\alpha X^{(2)}\right) + J \max(U^\alpha X^{(1)}, (1 - U)^\alpha X^{(2)})\right), \quad X^{(1)}, X^{(2)} \stackrel{i.i.d.}{\sim} \mu. \quad (3.17)$$

Observe that  $\delta_0$  is always a solution to (3.16), which is undesired. Hence, we have to choose a space that does not contain  $\delta_0$ . The operator is not a contraction in the Wasserstein metric. The Zolotarev metric [10] gives tighter bounds for proving contraction once the mean is fixed, but  $T_\alpha$  does not preserve the mean.

# A

## Appendix

### A.1 The Shift-Or algorithm

Let  $\Sigma$  be the alphabet,  $T$  be the text, and  $P$  be the pattern. Suppose that we have registers of length  $w$  bits. This requires that  $|P| \leq w$ . As usual, we index characters in the pattern  $P$  and  $T$  from left to right, starting from 0. But for the registers, it is convenient to index bits from right to left (LSB), starting from 0.

In the compile stage, we construct masks for each character in the alphabet, which we denote by a function  $M : \Sigma \rightarrow \{0, 1\}^w$ . For each character  $c \in \Sigma$ , we have

$$\begin{aligned} P[i] = c &\iff M[c][i] = 0, \quad i \in [0, |P| - 1] \\ &M[c][i] = 1, \quad i \in [|P|, w - 1] \end{aligned} \tag{1.1}$$

The execution stage is as follows. We initialize a state register  $S$  to all 1s ( $S \leftarrow \sim 0$ ). At each step  $i \geq 0$ , the register  $S$  is updated according to the following formula

$$S \leftarrow (S \ll 1) \mid M[T[i]]. \tag{1.2}$$

The algorithm leverages the fact that every time the state register is shifted to the left, a 0 is introduced at the least significant bit. This 0 is then propagated to the left as long as the characters in the text match the characters in the pattern. If a mismatch occurs, that 0 is replaced by a 1, and the propagation stops. The state register  $S$  keeps track of the longest prefix of the pattern that matches a suffix of the text ending at position  $i$ . At step  $i$ , if  $S[|P| - 1] = 0$ , then there is a match ending at position  $i$  in the text. Correctness is proved by using induction to check the invariant: at step  $i$ , we have

$$S[j] = 0 \iff T[i - j, i] = P[0, j], \forall j \in [0, |P| - 1]. \tag{1.3}$$

**Example A.1.** ■

### A.2 BST-like distributions of regular expression trees

**Lemma A.1 (Asymptotics of recurrence of full history sum)**

## A.3 Basics of Monte-Carlo simulation

## A.4 Remaining proofs

### A.4.1 Lemma 3.13

# B

## Bibliography

- [1] Victor Mikhaylovich Glushkov. “The abstract theory of automata”. In: *Russian Mathematical Surveys* 16.5 (1961), pp. 1–53.
- [2] Jan Holub, William F Smyth, and Shu Wang. “Fast pattern-matching on indeterminate strings”. In: *Journal of Discrete Algorithms* 6.1 (2008), pp. 37–50.
- [3] Donald E Knuth. *The Art of Computer Programming: Fundamental Algorithms, Volume 1*. Addison-Wesley Professional, 1997.
- [4] Florent Koechlin, Cyril Nicaud, and Pablo Rotondo. “Uniform random expressions lack expressivity”. In: *44th International Symposium on Mathematical Foundations of Computer Science (MFCS 2019)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2019, pp. 51–1.
- [5] Ralph Neininger and Ludger Rüschendorf. “A general limit theorem for recursive algorithms and combinatorial structures”. In: *The Annals of Applied Probability* 14.1 (2004), pp. 378–418.
- [6] Kun Qiu et al. “Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection”. In: *Proceedings of the 50th International Conference on Parallel Processing*. 2021, pp. 1–11.
- [7] Vectorcamp. *Vectorscan: A portable fork of the high-performance regular expression matching library*. <https://github.com/Vectorcamp/vectorscan>. 2023.
- [8] Xiang Wang et al. “Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}”. In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 2019, pp. 631–648.
- [9] Hao Xu et al. “Harry: A scalable SIMD-based multi-literal pattern matching engine for deep packet inspection”. In: *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE. 2023, pp. 1–10.
- [10] VM Zolotarev. “Approximation of distributions of sums of independent random variables with values in infinite-dimensional spaces”. In: *Theory of Probability & Its Applications* 21.4 (1977), pp. 721–737.